KP20 Beyond the SVM: The General Kernel Trick and the Kernel Perceptron

> Level: undergraduate, year 3
>
> Pattern Recognition | Chapter 4, §4.19

Slides: read and download

Learning objectives

- Write the three-step recipe of the kernel trick, use it to kernelize the Euclidean classifier and explain its relation to the Parzen estimate.
- Write the update and the discriminant of the kernel perceptron and work a small example by hand.
- Compare the hinge, exponential, logistic and 0/1 losses and explain why the logistic loss must be normalized.

KP20 Beyond the SVM: The General Kernel Trick and the Kernel Perceptron

One-sentence summary: The kernel trick is not only for the SVM; any linear algorithm that uses only inner products can be kernelized into a nonlinear version. The weight vector never has to be computed.

1. The three-step recipe

Plain explanation: Can any other algorithm that uses only inner products be lifted the way KP19 did?

- **Kernelization**: replacing the inner products of an algorithm by a kernel function, giving a nonlinear version in the original space.

The three-step recipe: (1) rewrite the algorithm with inner products only; (2) replace each inner product by K; (3) write the solution as a combination of training points. The weight vector is never computed, and K is as in KP19 and must meet the Mercer condition.

Figure `lec05-kp20-recipe`: the three-step recipe.



2. The Euclidean rule

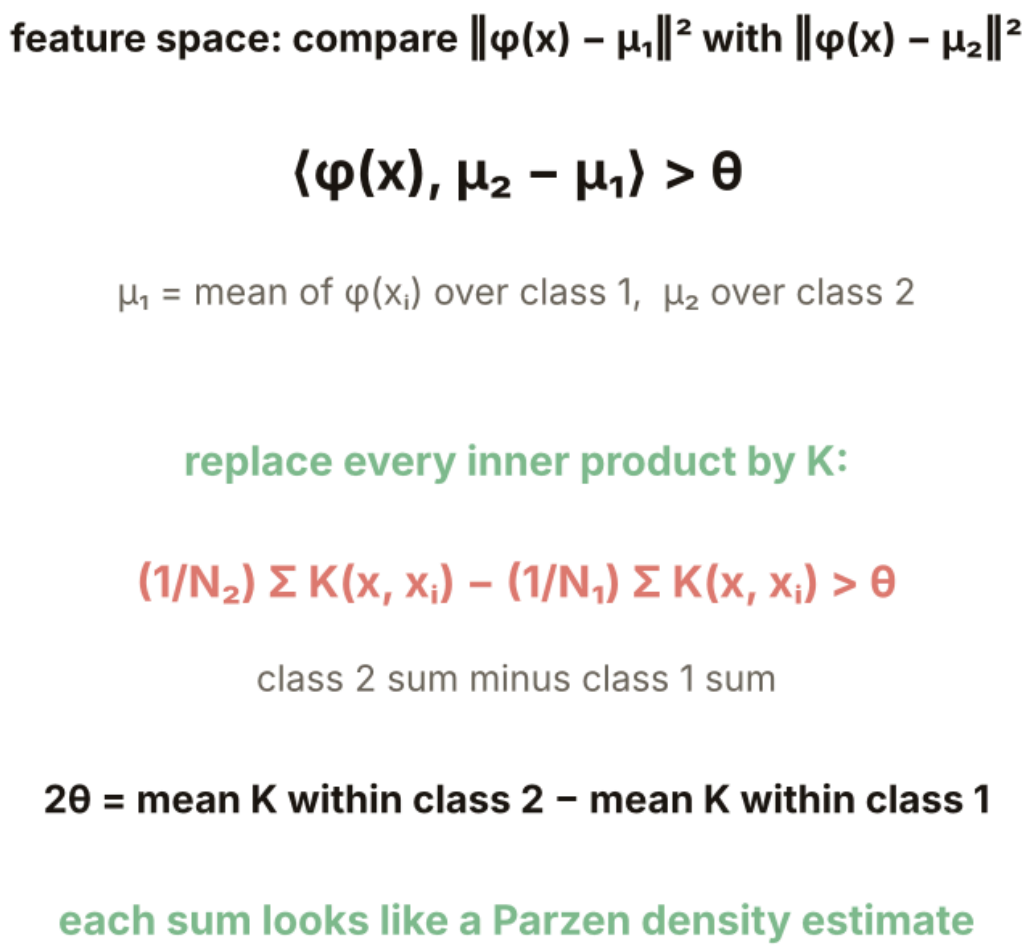
Formal definition: compare $\|\phi(x) - \mu_1\|^2$ and $\|\phi(x) - \mu_2\|^2$ in the kernel space, with μ_k the mean of $\phi(x_i)$ over class k; expanding and cancelling $\|\phi(x)\|^2$ leaves $\langle \phi(x), \mu_2 - \mu_1 \rangle > \theta$ with $\theta = (\|\mu_2\|^2 - \|\mu_1\|^2)/2$; true means class 2 ($y = -1$). With every inner product replaced by a kernel:

$$\frac{1}{N_2} \sum_{i:y_i=-1} K(x, x_i) - \frac{1}{N_1} \sum_{i:y_i=+1} K(x, x_i) > \theta,$$

where 2θ is the mean kernel value within class 2 minus that within class 1. The two sums look like Parzen density estimates (KP18), and are exactly that with the

Gaussian kernel.

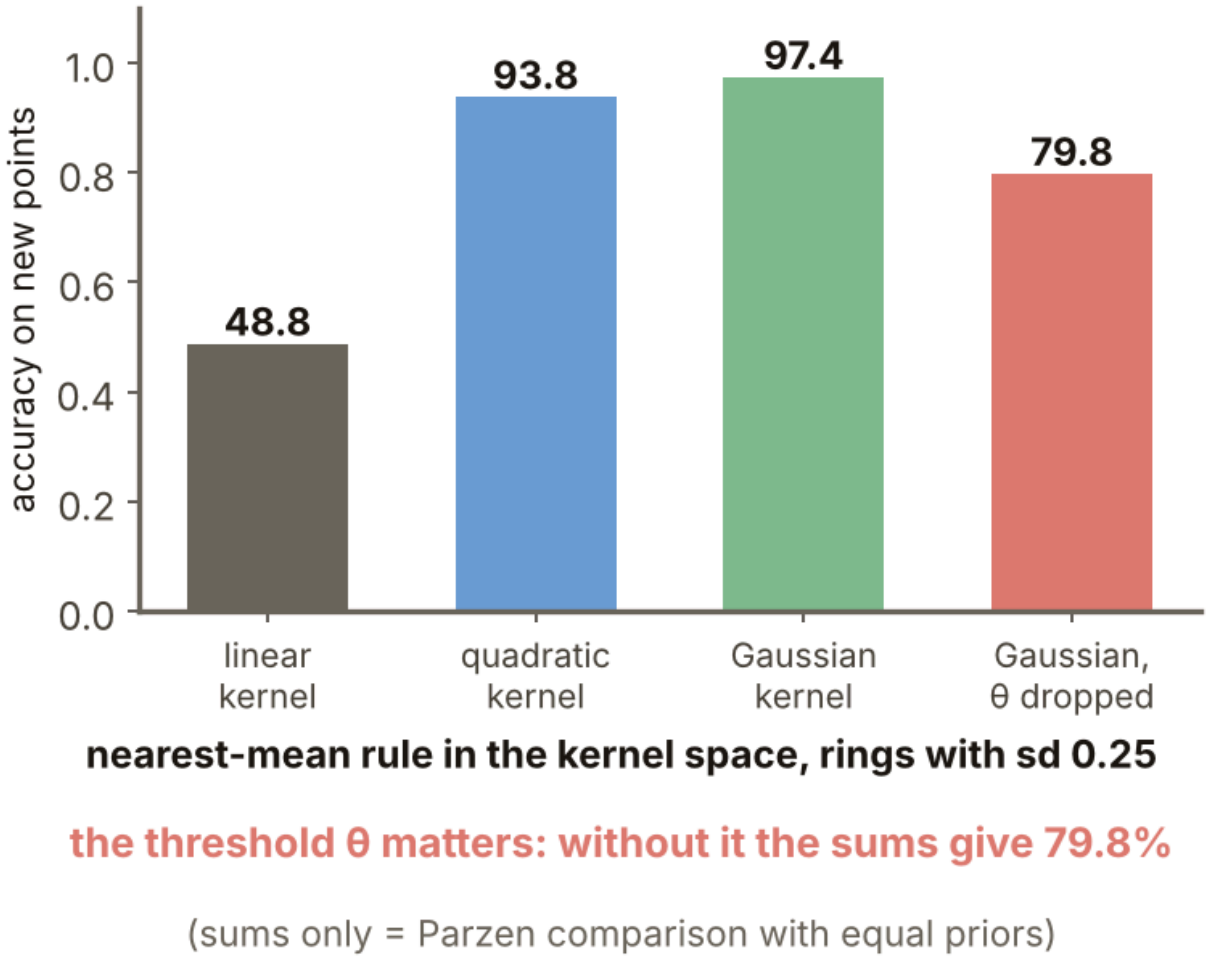
Figure `lec05-kp20-kmean`: the kernelized Euclidean classifier.



3. Comparison on the rings

Example (designed here): rings of radius 1 and 2 (sd 0.25), 60 training and 500 new points per class; Gaussian $\gamma = 0.5$, quadratic $(x^T z + 1)^2$. Accuracy on new points: linear 48.8%; quadratic 93.8%; Gaussian 97.4%. Dropping θ, comparing only the two sums as an equal-prior Parzen comparison, leaves 79.8%. The classes differ in mean kernel value within themselves and θ corrects for it, so "≈ Parzen" cannot ignore it. **Common misconception**: the two sums look like Parzen estimates, so comparing them directly is a Parzen classifier; without θ the result is far worse.

Figure `lec05-kp20-kmeanres`: new-point accuracy of the four versions.



4. The kernel perceptron

Kernel perceptron: a perceptron run in the kernel space that adds 1 to the counter α_i of a misclassified point instead of computing w. The update: on $y_t g(x_t) \le 0$, $w \leftarrow w + y_t \phi(x_t)$; from zero this ends as $w = \sum \alpha_i y_i \phi(x_i)$ and $w_0 = \sum \alpha_i y_i$, so $g(x) = \sum \alpha_i y_i K(x, x_i) + \sum \alpha_i y_i$. The algorithm: all $\alpha_i = 0$, sweep the training points pass after pass, add 1 to α_i when $y_i g(x_i) \le 0$, until a whole pass has no mistake. A linear perceptron converges only if the classes are linearly separable; after a kernel map to high dimension Cover's theorem (KP14) makes separability likely.

Figure `lec05-kp20-kperc`: the update, the discriminant and the algorithm.

on a mistake: $w \leftarrow w + y_t\,\varphi(x_t)$

count the mistakes of each point in α_i

$$w = \Sigma\,\alpha_i\,y_i\,\varphi(x_i),\quad w_0 = \Sigma\,\alpha_i\,y_i$$

$$g(x) = \Sigma\,\alpha_i\,y_i\,K(x, x_i) + \Sigma\,\alpha_i\,y_i$$

start with all $\alpha_i = 0$

repeat: for i = 1 … N

if $y_i\,g(x_i) \le 0$ then $\alpha_i \leftarrow \alpha_i + 1$

until no mistakes in a whole pass

w itself is never computed

5. A hand trace

Example (designed here): points $(1,0)+$, $(-1,0)+$, $(0,1)-$, $(0,-1)-$ with $K = (x^T z + 1)^2$; no straight line separates them. Pass 1: points 1, 3 and 4 are wrong, $\alpha = (1,0,1,1)$; pass 2: point 2 is wrong, $\alpha = (1,1,1,1)$; pass 3 has no mistake, stop. Summing the four kernels gives $g(x) = 2(x_1^2 - x_2^2)$, which is $+8$ at $(2,0)$ and -8 at $(0,2)$.

Figure `lec05-kp20-trace`: the mistakes and α of each pass.

points (1,0)+, (−1,0)+, (0,1)−, (0,−1)−; $K = (x^{\mathsf{T}}z + 1)^2$

pass	mistakes	α after the pass
1	3	(1, 0, 1, 1)
2	1	(1, 1, 1, 1)
3	0	(1, 1, 1, 1)

$$g(x) = \Sigma\, \alpha_i\, y_i\, K(x, x_i) + \Sigma\, \alpha_i\, y_i = 2\,(x_1^2 - x_2^2)$$

no straight line separates these four points

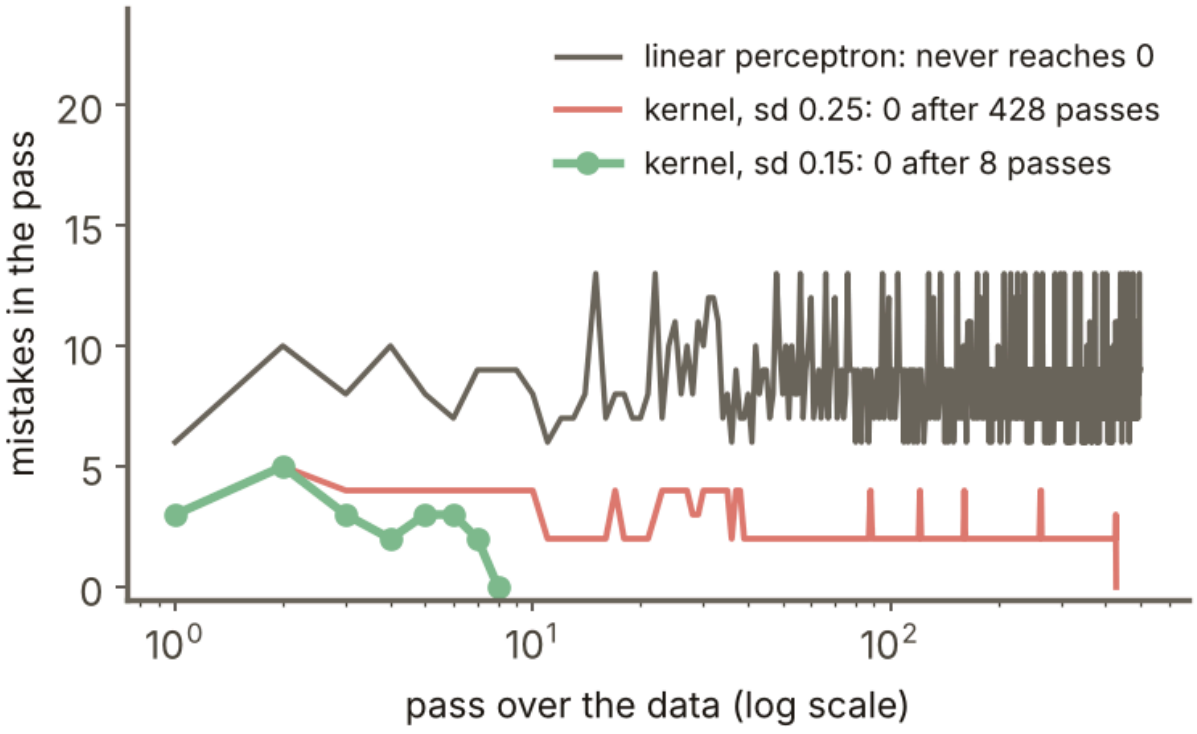
but g has the right sign on all four: g = +8 at (2, 0)

g = −8 at (0, 2)

6. The rings experiment

Example (designed here): rings, 60 training and 500 new points per class; Gaussian $\gamma = 0.5$; sd 0.15 (separable) and 0.25 (touching at the edges). The linear perceptron still errs after 500 passes with 50.0% on new points. Kernel perceptron: sd 0.15 stops after 8 passes with 98.2%; sd 0.25 after 428 with 95.6% and 22 points with $\alpha > 0$. It stops in finitely many passes when separable but needs many more when the classes sit close; for sd 0.25 and one random draw, $\gamma = 0.1, 2, 8$ take 191, 571 and 175 passes.

Figure `lec05-kp20-epochs`: mistakes per pass (log-scale horizontal axis).



linear: 50.0% on new points after 500 passes

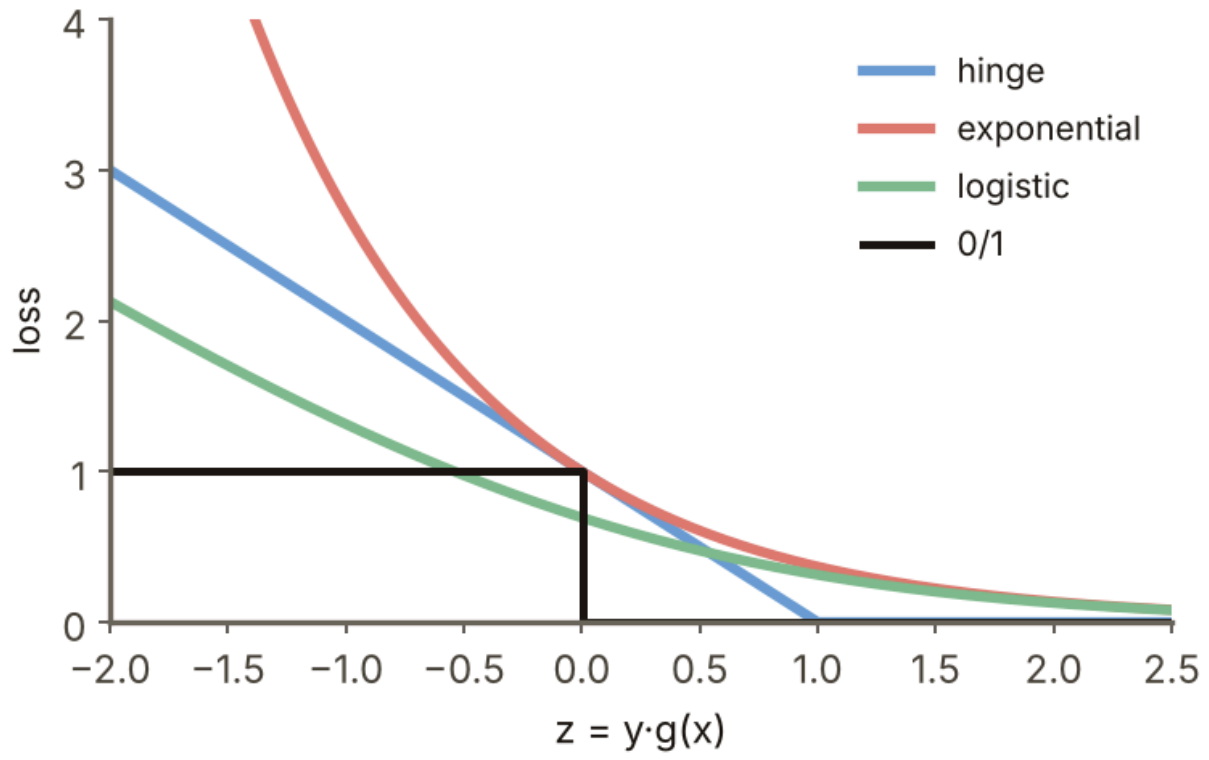
kernel (γ = 0.5): 98.2% (sd 0.15), 95.6% (sd 0.25)

touching classes need many more passes to separate

7. Loss functions

Margin value $z = y\,g(x)$: $z > 0$ is correct, $z \leq 0$ is a mistake, and the more negative the worse. A **loss function** gives each z a penalty and sets the personality of an algorithm. The four: hinge $\max(0, 1 - z)$; exponential e^{-z}; logistic $\ln(1 + e^{-z})$; 0/1 (1 when $z \leq 0$). Hinge is the SVM soft margin; exponential is used by AdaBoost (KP25); logistic is the negative log-likelihood of the logistic probability model; 0/1 is not differentiable.

Figure `lec05-kp20-losses`: the four losses against z.



z > 0: classified correctly; z ≤ 0: a mistake

hinge max(0,1−z), exponential e^(−z), logistic ln(1+e^(−z))

the exponential is the steepest on the left

8. The personality of a loss

At $z = -2$: hinge 3, exponential 7.389, logistic 2.127, 0/1 equal to 1; at $z = 0$: 1, 1, 0.693, 1; at $z = 2$: 0, 0.135, 0.127, 0. The exponential is steepest on the negative side and one badly wrong point costs $7.389/3 = 2.46$ times the hinge loss; this drives AdaBoost and makes it sensitive to outliers (the source calls it a weakness on high-Bayes-error tasks, see KP25). **To remember**: the general recipe is (1) inner products only, (2) replace them by K, (3) express the solution as a combination of training points; the weight vector is never computed.

Figure `lec05-kp20-lossvals`: the four losses at five values of z.

z	-2	-1	0	1	2
hinge	3	2	1	0	0
exponential	7.389	2.718	1	0.368	0.135
logistic	2.127	1.313	0.693	0.313	0.127
0/1	1	1	1	0	0

at z = −2: exponential 7.389 against hinge 3

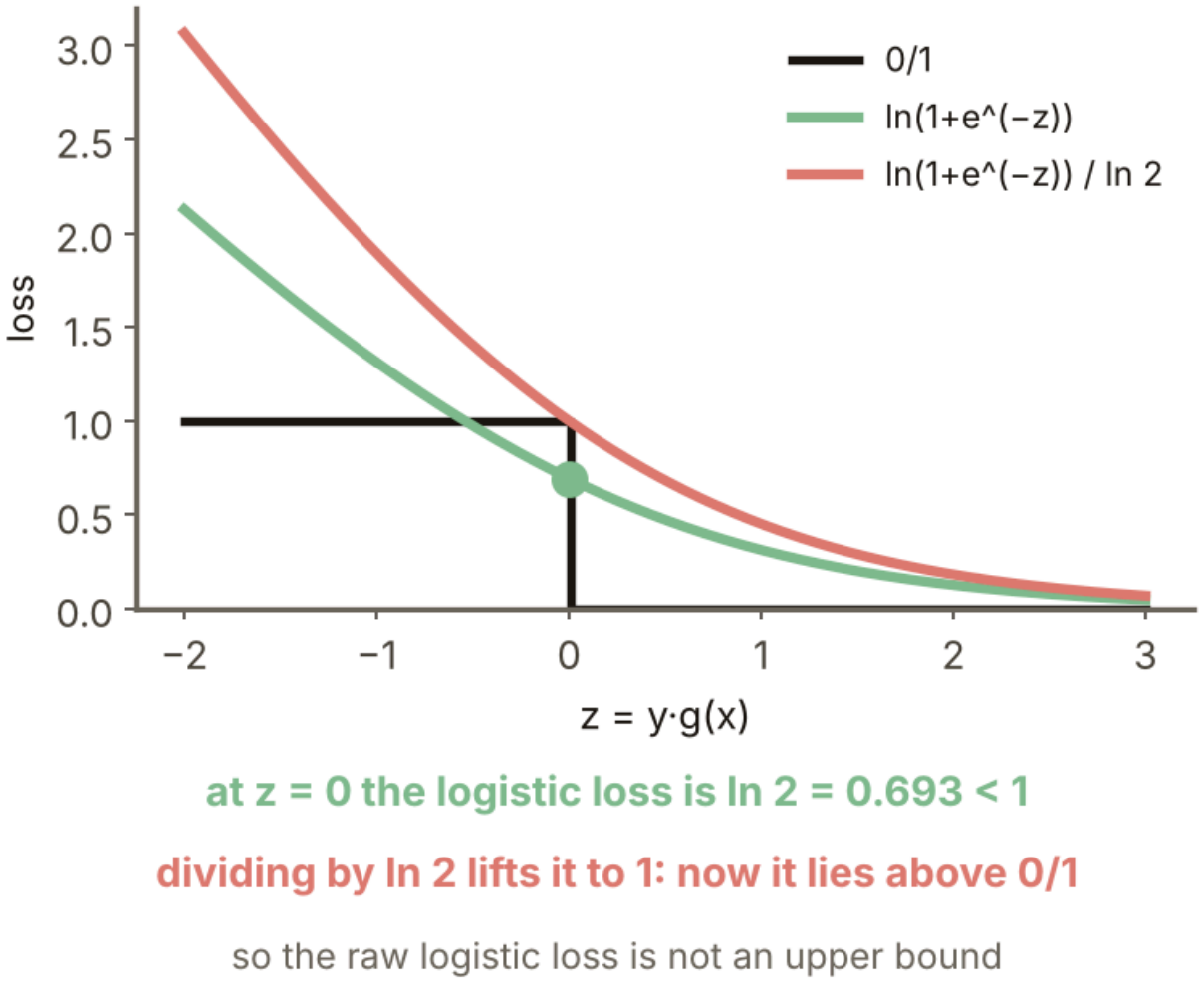
one badly wrong point weighs 2.5 times more

(7.389 / 3 = 2.46)

Extension: the logistic bound, practice and worked solution

The logistic loss needs normalizing: hinge and exponential are never below 0/1; the logistic $\ln(1+e^{-z})$ is only $\ln 2 = 0.693 < 1$ at $z = 0$, so it is not an upper bound. Dividing by $\ln 2$ makes it exactly 1 at $z = 0$ and at least 1 for $z \leq 0$, a convex upper bound of 0/1 (the source's normalized logistic). Check: $z = -2$: $2.127/0.693 = 3.069$; $z = -1$: $1.313/0.693 = 1.895$.

Figure `lec05-kp20-logup`: 0/1, logistic and logistic divided by $\ln 2$.



Practice: points $(1,0)+$, $(-1,0)+$, $(0,1)-$, $(0,-1)-$, $K=(x^Tz+1)^2$, $\alpha=(1,1,1,1)$.

1. Compute $g(x)$ at $x=(1,2)$ from the four kernels; which class?
2. Simplify g to a formula in x_1,x_2 and check question 1.

Figure `lec05-kp20-practice`: the setup of the practice.

points (1,0)+, (−1,0)+, (0,1)−, (0,−1)−

$K = (x^T z + 1)^2$, all $\alpha_i = 1$

question 1: g(x) at x = (1, 2) from the four kernels

question 2: simplify g to a formula in x_1, x_2

$$g(x) = \Sigma\, \alpha_i\, y_i\, K(x, x_i) + \Sigma\, \alpha_i\, y_i$$

Worked solution of questions 1 and 2: (1) $K(x, (1,0)) = 4$, $K(x, (-1,0)) = 0$, $K(x, (0,1)) = 9$, $K(x, (0,-1)) = 1$; $\sum \alpha_i y_i = 0$; $g = 4 + 0 - 9 - 1 + 0 = -6$, the negative class. (2) Adding the four squares, $g = 2(x_1^2 + 1) - 2(x_2^2 + 1) = 2(x_1^2 - x_2^2)$; at $x = (1, 2)$: $2(1 - 4) = -6$, the same as question 1. (3) Check: $g > 0$ means $|x_1| > |x_2|$, the positive class; the boundary is two diagonal lines, and the four points no line could split are separated.

Figure `lec05-kp20-practice-sol`: the computation both ways.

$$K(x,(1,0)) = (1+1)^2 = 4, \quad K(x,(-1,0)) = 0$$

$$K(x,(0,1)) = (2+1)^2 = 9, \quad K(x,(0,-1)) = 1$$

$$\Sigma\, \alpha_i y_i = 1 + 1 - 1 - 1 = 0$$

$$g = 4 + 0 - 9 - 1 + 0 = -6 \;\rightarrow\; \text{class } -$$

$$\text{sum of the four squares} = 2(x_1^2+1) - 2(x_2^2+1)$$

$$g(x) = 2\,(x_1^2 - x_2^2) = 2\,(1 - 4) = -6$$

$$\text{both ways give } -6$$

Terms and source

中文	English
核化	Kernelization
核感知器	Kernel perceptron
間隔值	Margin value
損失函數	Loss function

Source: Chapter 4, KP20, §4.19 (read 2026-10-25). The three-step recipe, the kernelized Euclidean classifier and its relation to Parzen, the kernel perceptron update and discriminant, the four losses and their uses and the 'to remember' line follow the source; the rings comparison, the 4-point hand run, the passes and accuracies, the loss table, the logistic normalization and the practice were designed and recomputed for this lecture ([added], (program-computed)).